

Information Retrieval

Index Construction

Previous lessons

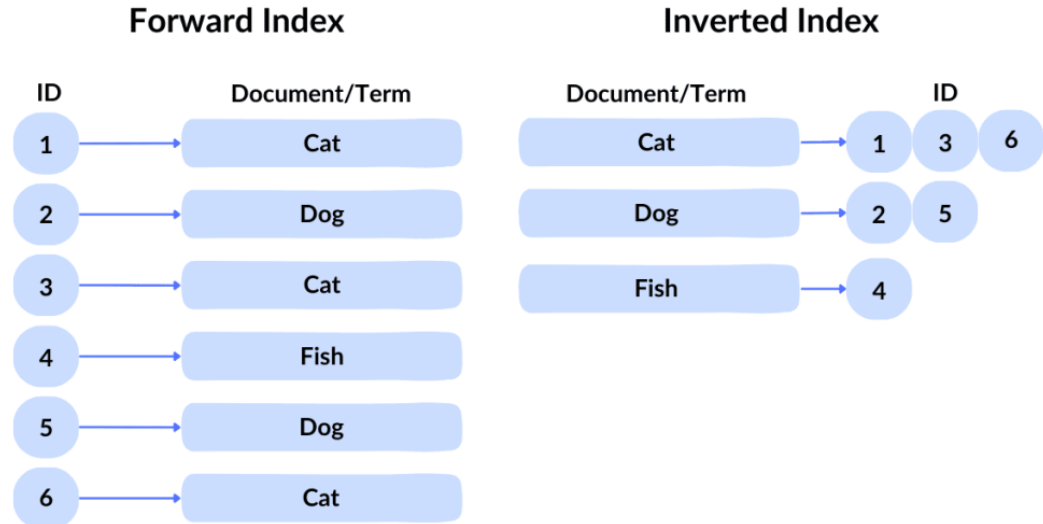
- ❖ Boolean indexing and search
- ❖ Term-document matrix
- ❖ Inverted index

- ❖ **Documents**
- ❖ **Tokenization**
- ❖ **Token processing: stopping, normalization, stemming, lemmatization**

- ❖ Handling phrases: bigram index, positional index

This lesson

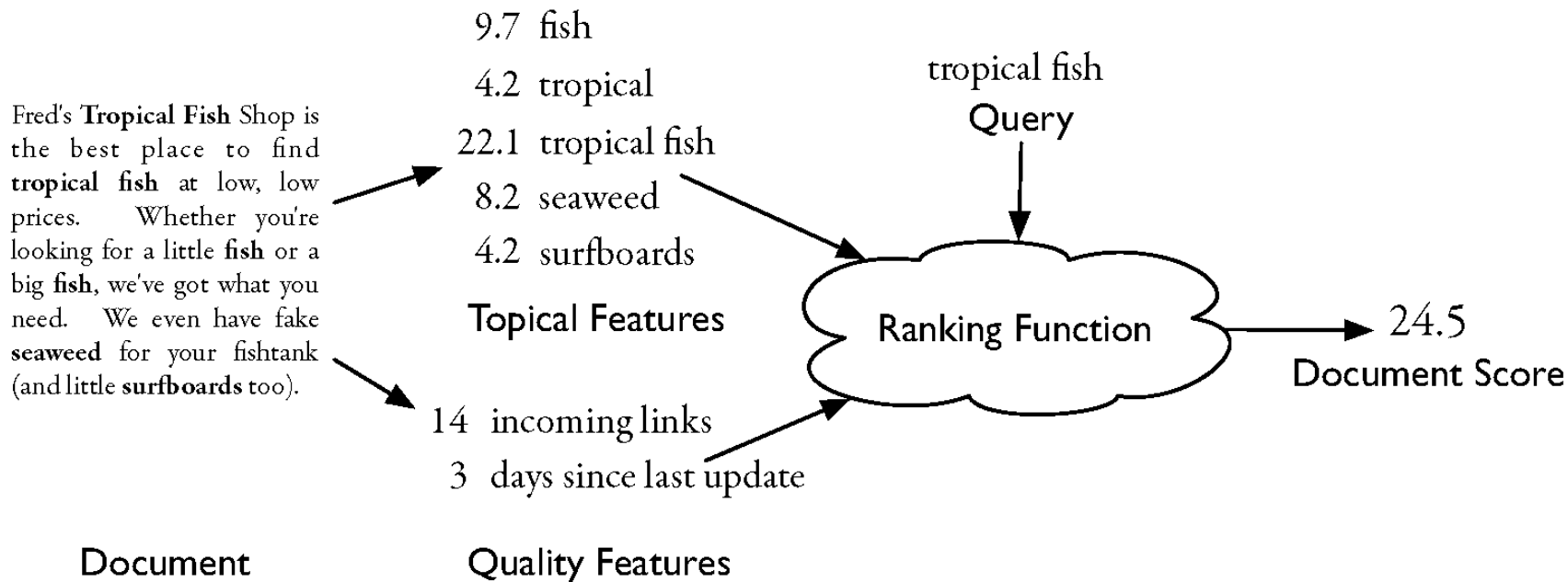
- ❖ Index construction
- ❖ Data structures
- ❖ Indexing strategies
- ❖ Distributed indexing



Indexes

- ❖ **Indexes** are data structures designed to make searches faster
- ❖ Text search engines use a particular form of search: **ranking**
 - documents are retrieved in sorted order according to a score computed using the document representation, the query, and a ranking algorithm
- ❖ Requires a specific data structure: the most common is the **inverted index**
 - general name for a class of structures
 - “inverted” because documents are associated with words, rather than words with documents

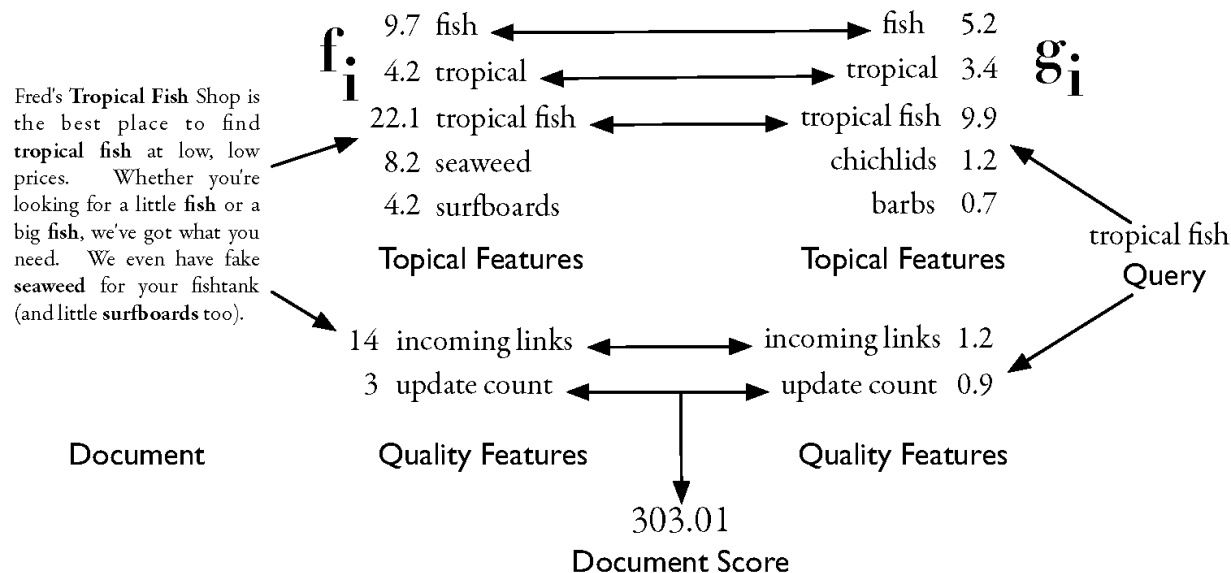
Abstract Model of Ranking



More Concrete Model

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

f_i is a document feature function
 g_i is a query feature function



Inverted Index

- ❖ Each index term is associated with an inverted list
 - Contains **lists of word** occurrences in documents, and other information
 - Each entry is called a **posting**
 - The part of the posting that refers to a specific document or location is called a pointer
 - Each document in the collection is given a **unique number**
 - Lists are usually **document-ordered** (sorted by document number)

Simple Inverted Index

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

Collection



index term	posting list	
and	1	only 2
aquarium	3	pigmented 4
are	3 4	popular 3
around	1	refer 2
as	2	referred 2
both	1	requiring 2
bright	3	salt 1 4
coloration	3 4	saltwater 2
derives	4	species 1
due	3	term 2
environments	1	the 1 2
fish	1 2 3 4	their 3
fishkeepers	2	this 4
found	1	those 2
fresh	2	to 2 3
freshwater	1 4	tropical 1 2 3
from	4	typically 4
generally	4	use 2
in	1 4	water 1 2 4
include	1	while 4
including	1	with 2
iridescence	4	world 1
marine	2	
often	2 3	

Simple Inverted Index

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

❖ Query:
– Tropical Fish

and	1				only	2
aquarium	3				pigmented	4
are	3	4			popular	3
around	1				refer	2
as	2				referred	2
both	1				requiring	2
bright	3				salt	1 4
coloration	3	4			saltwater	2
derives	4				species	1
due	3				term	2
environments	1				the	1 2
fish	1	2	3	4	their	3
fishkeepers	2				this	4
found	1				those	2
fresh	2				to	2 3
freshwater	1	4			tropical	1 2 3
from	4				typically	4
generally	4				use	2
in	1	4			water	1 2 4
include	1				while	4
including	1				with	2
iridescence	4				world	1
marine	2					
often	2	3				

Simple Inverted Index

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

❖ Issue

- does not record the **number of times** each word appears
- no reason to prefer any of these sentences over any other

and	1				only	2
aquarium	3				pigmented	4
are	3	4			popular	3
around	1				refer	2
as	2				referred	2
both	1				requiring	2
bright	3				salt	1 4
coloration	3	4			saltwater	2
derives	4				species	1
due	3				term	2
environments	1				the	1 2
fish	1	2	3	4	their	3
fishkeepers	2				this	4
found	1				those	2
fresh	2				to	2 3
freshwater	1	4			tropical	1 2 3
from	4				typically	4
generally	4				use	2
in	1	4			water	1 2 4
include	1				while	4
including	1				with	2
iridescence	4				world	1
marine	2					
often	2	3				

Inverted Index with counts

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

- ❖ Query:
 - tropical fish
 - S_2 should be preferred over S_1 and S_3
- ❖ Supports better ranking algorithms

and	1:1					only	2:1				
aquarium	3:1					pigmented	4:1				
are	3:1	4:1				popular	3:1				
around	1:1					refer	2:1				
as	2:1					referred	2:1				
both	1:1					requiring	2:1				
bright	3:1					salt	1:1	4:1			
coloration	3:1	4:1				saltwater	2:1				
derives	4:1					species	1:1				
due	3:1					term	2:1				
environments	1:1					the	1:1	2:1			
fish	1:2	2:3	3:2	4:2		their	3:1				
fishkeepers	2:1					this	4:1				
found	1:1					those	2:1				
fresh	2:1					to	2:2	3:1			
freshwater	1:1	4:1				tropical	1:2	2:2	3:1		
from	4:1					typically	4:1				
generally	4:1					use	2:1				
in	1:1	4:1				water	1:1	2:1	4:1		
include	1:1					while	4:1				
including	1:1					with	2:1				
iridescence	4:1					world	1:1				
marine	2:1										
often	2:1	3:1									

Posting → a:b
a: document ID
b: # the word appears in 'a'

Inverted Index with positions

one posting per word occurrence

- S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.
- S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.
- S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.
- S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

- ❖ Query:
 - tropical fish

tropical

1,1
1,2

1,7

2,6
2,7

2,17
2,18

2,23

3,1
3,2

 3,6 4,3 4,13

- Supports proximity matches

and	1,15					marine	2,22		
aquarium	3,5					often	2,2	3,10	
are	3,3	4,14				only	2,10		
around	1,9					pigmented	4,16		
as	2,21					popular	3,4		
both	1,13					refer	2,9		
bright	3,11					referred	2,19		
coloration	3,12	4,5				requiring	2,12		
derives	4,7					salt	1,16	4,11	
due	3,7					saltwater	2,16		
environments	1,8					species	1,18		
fish	1,2	1,4	2,7	2,18	2,23	term	2,5		
			3,2	3,6	4,3	the	1,10	2,4	
			4,13			their	3,9		
fishkeepers	2,1					this	4,4		
found	1,5					those	2,11		
fresh	2,13					to	2,8	2,20	3,8
freshwater	1,14	4,2				tropical	1,1	1,7	2,6
from	4,8					typically	4,6		2,17
generally	4,15					use	2,3		3,1
in	1,6	4,1				water	1,17	2,14	4,12
include	1,3					while	4,10		
including	1,12					with	2,15		
iridescence	4,9					world	1,11		

Posting → a:b
a: document ID
b: the word position in 'a'

Fields and Extents

❖ Document structure is useful in the search

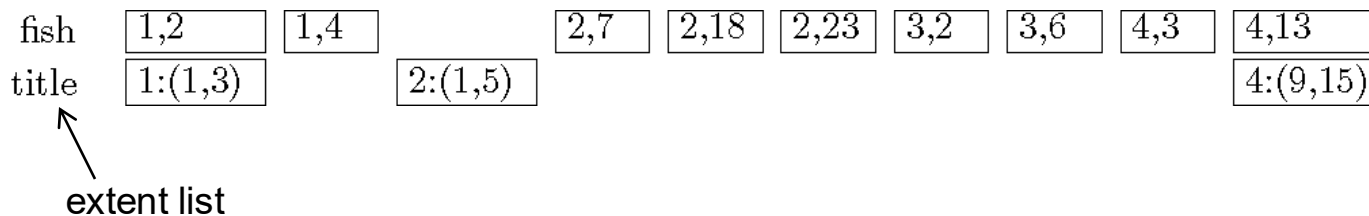
- field restrictions
 - e.g., date, from:
- some fields are more important
 - e.g., title, headings

❖ Options:

- separate index (set of inverted lists) for each field type
 - e.g., one for title and one for body
 - Problem: General search must read multiple indexes
- use extent lists

Extent Lists

- ❖ The traditional Inverted Index must be extended to store not just the Document ID, but also the **Field ID**.
 - Field Indexing: Each document is parsed, and its content is separated into distinct fields (e.g., title, body, author, date).
 - Structured Posting Lists: The posting list for each term is extended to include the field where the term was found, and sometimes even the position within that field.



- ❖ The title in document 1 starts from a word in position 1 and ends before position 3
 - fish in the title of document 1

Precomputed scores in inverted index

- ❖ Normally, the Posting List stores document IDs and possibly word positions. To rank documents, the ranking score is calculated at query time.
- ❖ Precomputed scores change this by calculating the core relevance value for each term-document pair during index construction and storing it directly in the posting list.
 - e.g., list for “fish” [(1:3.6), (3:2.2)], where 3.6 is total feature value for document 1
- ❖ Advantage (Speed): Because the system avoids many complex calculations during the search
 - often just needing to sum up precomputed scores for multi-term queries.
- ❖ Disadvantage (Flexibility): It reduces flexibility because the score is fixed.
 - Difficult to incorporate dynamic factors (like user location or current time) into this score.
- ❖ Phrase Information Loss: When the score is precomputed, phrase information is often lost.
 - Since the score is for the single term “fish” across the document, the positional data needed to verify a phrase like “tropical fish” is usually omitted to save space.

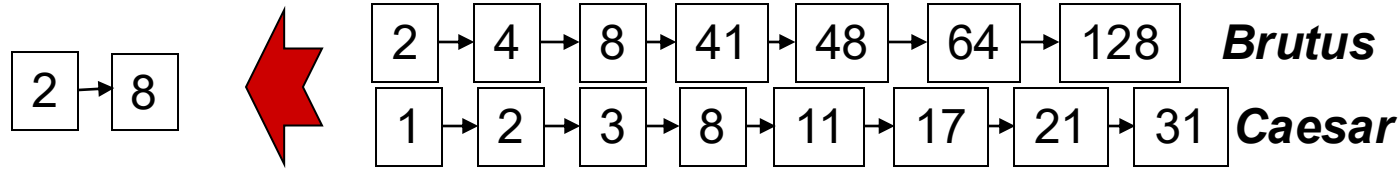
Score-ordered lists (not document-ordered)

- ❖ The key idea is to **sort a term's posting list by its precomputed relevance score** rather than the traditional Document ID.
 - This optimization is only possible when a document's relevance score for a specific term is calculated before query time and stored in the index.
- ❖ The fundamental prerequisite is that the posting list for each term must contain a precalculated score (a "total feature value") for every document it appears in. For example, the list for "fish" might be
 - [(1: 3.6), (3: 2.2)], where 3.6 is the precomputed score for document 1.
- ❖ Instead of sorting the list by Document ID (1, 3, 5, 7...), the list is sorted by the score in descending order (e.g., 3.6, 2.2, 1.5, ...).
- ❖ Efficient Query Processing: Since the list is sorted by relevance, the query processing engine can employ a technique called "early termination."
 - The engine only needs to look at the top part of each inverted list, where the highest-scoring documents are recorded.
 - Once the system has found the required number of top results (e.g., the top 10), it can stop processing the rest of the list, saving significant time.
- ❖ Very efficient for single-word queries.
 - For a query like "fishing," the system goes directly to the "fishing" list and reads the top-scoring documents, which are already guaranteed to be the most relevant based on the precomputed score.

Faster postings merges: Skip pointers/Skip lists

Recall basic merge

- ❖ Walk through the two postings simultaneously, in time linear in the total number of postings entries

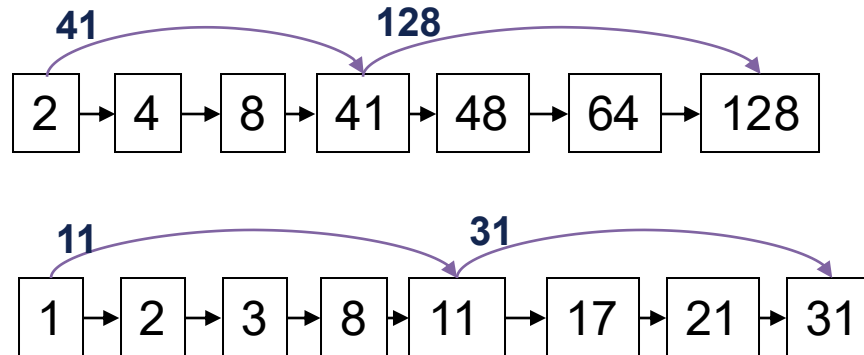


- ❖ If the list lengths are m and n , the merge takes $O(m+n)$ operations.
- ❖ Can we do better?

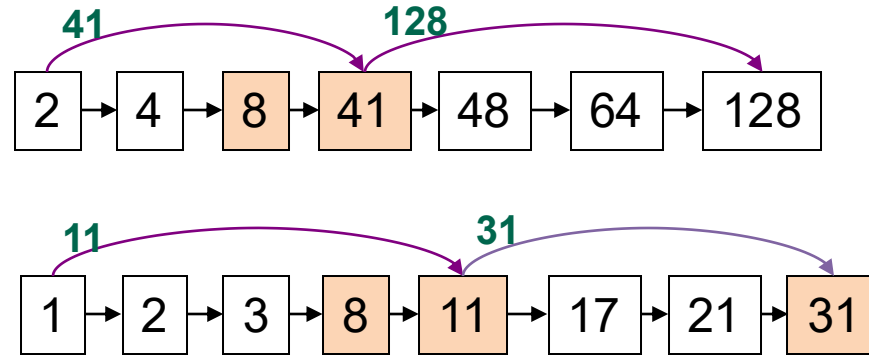
Augment postings with skip pointers (at indexing time)

❖ Why?

- To skip postings that will not figure in the search results.



Query processing with skip pointers

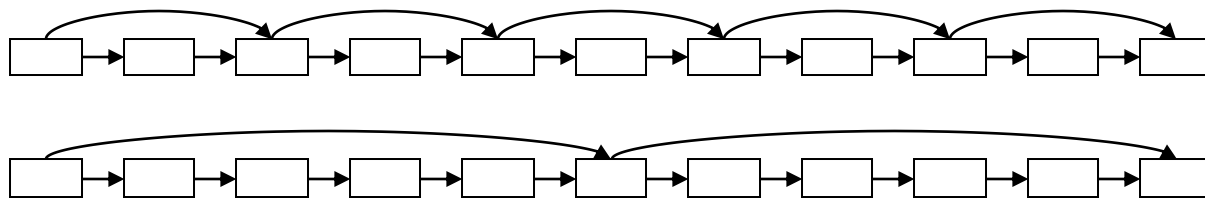


- Suppose we've stepped through the lists until we process 8 on each list.
 - We match it and advance.
- We then have **41** and **11** on the lower. **11** is smaller.
- But the skip successor of **11** on the lower list is **31**, so we can skip ahead past the intervening postings.

Placing skips

❖ Tradeoff:

- More skips $\rightarrow$ shorter skip spans $\Rightarrow$ more likely to skip.
 - But lots of comparisons to skip pointers.
- Fewer skips $\rightarrow$ few pointer comparison,
 - but then long skip spans $\Rightarrow$ few successful skips.



❖ Simple heuristic:

- for postings of length L , use $\sqrt{L}$ evenly-spaced skip pointers.
- Easy if the index is relatively static; harder if L keeps changing because of updates.

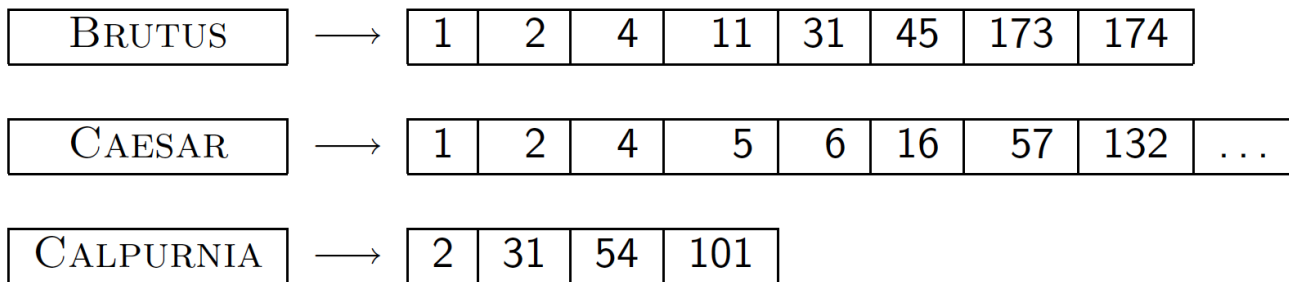
Postings intersection with skip pointers

```
INTERSECTWITHSKIPS( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then if  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
9          then while  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
10             do  $p_1 \leftarrow \text{skip}(p_1)$ 
11             else  $p_1 \leftarrow \text{next}(p_1)$ 
12      else if  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
13          then while  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
14             do  $p_2 \leftarrow \text{skip}(p_2)$ 
15             else  $p_2 \leftarrow \text{next}(p_2)$ 
16  return  $answer$ 
```

Index Construction

Dictionary data structures for inverted indexes

- ❖ The dictionary data structure stores the term vocabulary, document frequency, pointers to each postings list ...
 - in what data structure?



Dictionary data structures

- Sorted array:

- Binary search if can be kept in memory
- High overhead for additions

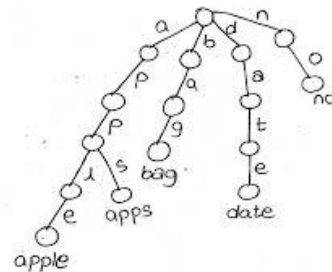
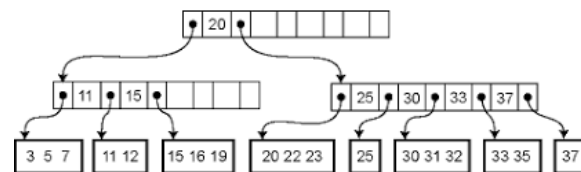
 Hashing

- Fast look-up
- Collisions

- Binary tree (BST), B-Tree, B+Tree, Trie, ...

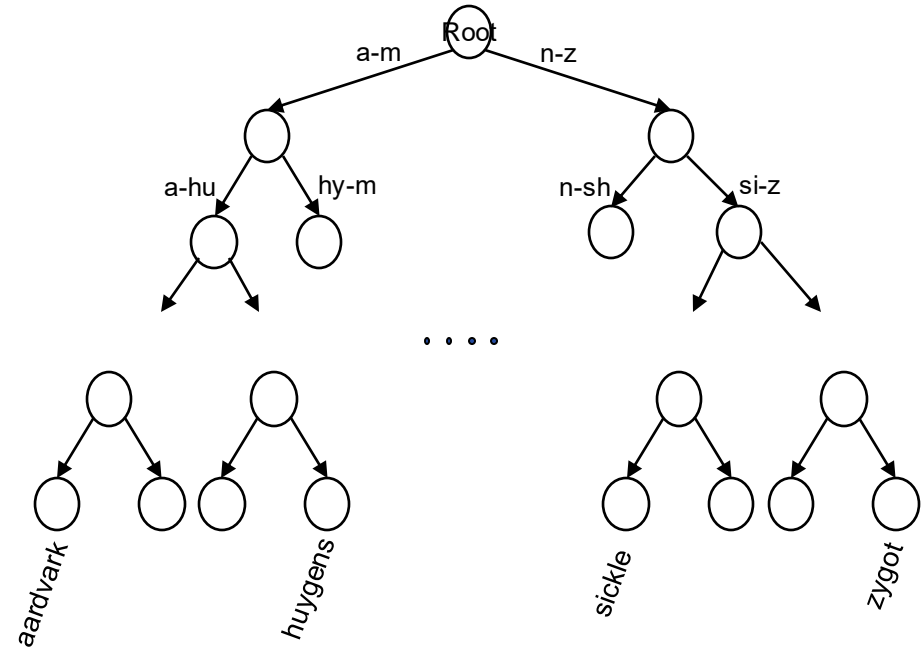
- Maintain balance - always log look-up time
- Can insert and delete

- Some IR systems use trees, some hashes



Balanced BST

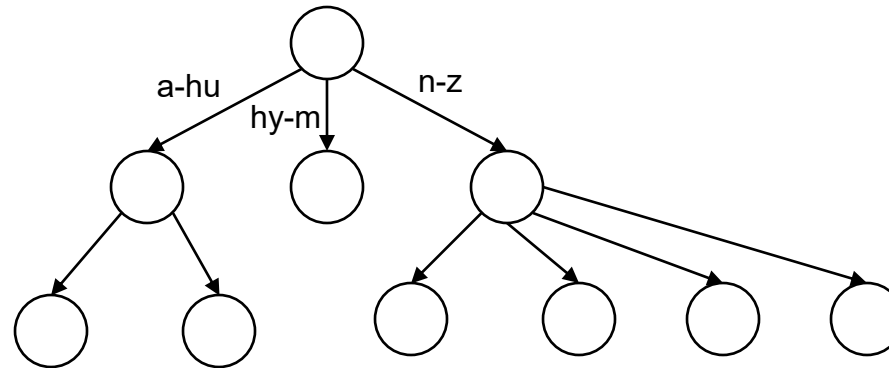
- ❖ BST requires $O(\log_2 n)$ time in the average case
 - needs $O(n)$ time in the worst case, when the unbalanced tree resembles a linked list (degenerate tree)
- ❖ To keep the BST balanced we need to verify balance for each insert and delete
- ❖ BST is balanced if:
 - the difference between left and right branches is 0 or 1
 - all sub-tree are balanced



Tree: B-tree

❖ Definition:

- Every internal node has several children in the interval $[a,b]$ where a, b are appropriate natural numbers, e.g., $[2,4]$.

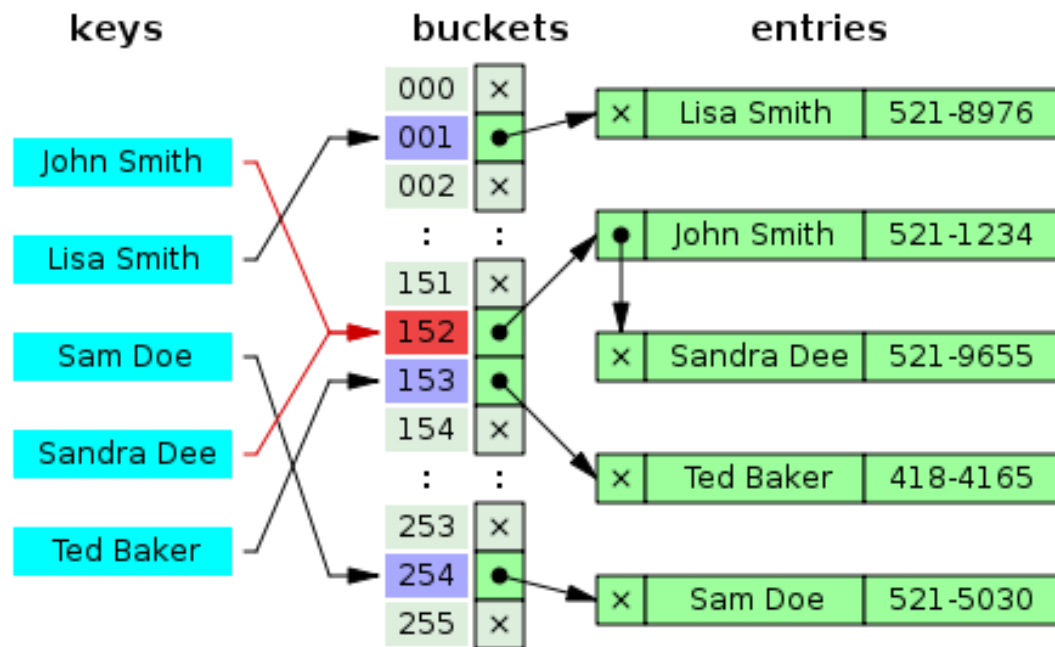


Trees

- ❖ Simplest: binary tree
- ❖ More usual: B-trees
- ❖ Trees require ordering of strings

- ❖ Pros:
 - Solves the prefix problem
 - e.g., terms starting with "*univ*"
- ❖ Cons:
 - BST are slower: $O(\log_2 N)$
 - and this requires balanced tree
 - Rebalancing trees is expensive
 - but B-trees mitigate the rebalancing problem

Hash Table



Hashes

- ❖ Each vocabulary term is hashed to an integer
- ❖ Pros:
 - Lookup is faster than for a tree: $O(1)$
- ❖ Cons:
 - No prefix search
 - No easy way to find minor variants:
 - judgment/judgement
 - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing everything

Index construction – A simple approach

```
procedure BUILDINDEX( $D$ )  
   $I \leftarrow \text{HashTable}()$   
   $n \leftarrow 0$   
  for all documents  $d \in D$  do  
     $n \leftarrow n + 1$   
     $T \leftarrow \text{Parse}(d)$   
    Remove duplicates from  $T$   
    for all tokens  $t \in T$  do  
      if  $I_t \notin I$  then  
         $I_t \leftarrow \text{Array}()$   
      end if  
       $I_t.\text{append}(n)$   
    end for  
  end for  
  return  $I$   
end procedure
```

- ▷ D is a set of text documents
- ▷ Inverted list storage
- ▷ Document numbering

- ▷ Parse document into tokens

Parse each document
into tokens

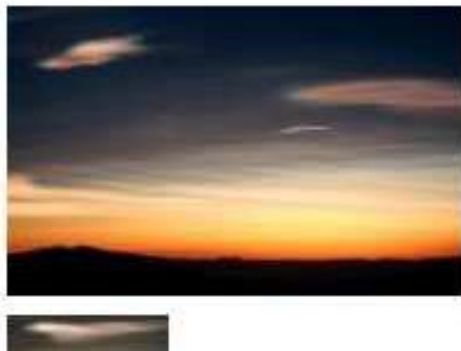
- Requires all the inverted lists in memory
- Is sequential, with no obvious way to parallelize its construction

Dataset example: Reuters RCV1 collection

- ❖ Contains one year of Reuters newswire
 - between August 20, 1996 and August 19, 1997
- ❖ Typical document:

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET



[Email This Article](#) | [Print This Article](#) | [Reprints](#)

[\[-\]](#) [Text](#) [\[+\]](#)

SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian meteorological base at Mawson Station on July 25.

Reuters RCV1 statistics

symbol	statistic	value
N	documents	800,000
L	avg. # tokens per doc	200
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
M	terms (= word types)	400,000
	avg. # bytes per term	7.5
	larger than the size of an average token because it represents the stored, normalized word type.	
T	non-positional postings	100,000,000

Recall index construction

- ❖ Documents are parsed to extract words and these are saved with the Document ID

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: Sort

- ❖ After all documents have been parsed, the inverted file is sorted by terms.

We focus on this sort step.
We have 100M items to sort.

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



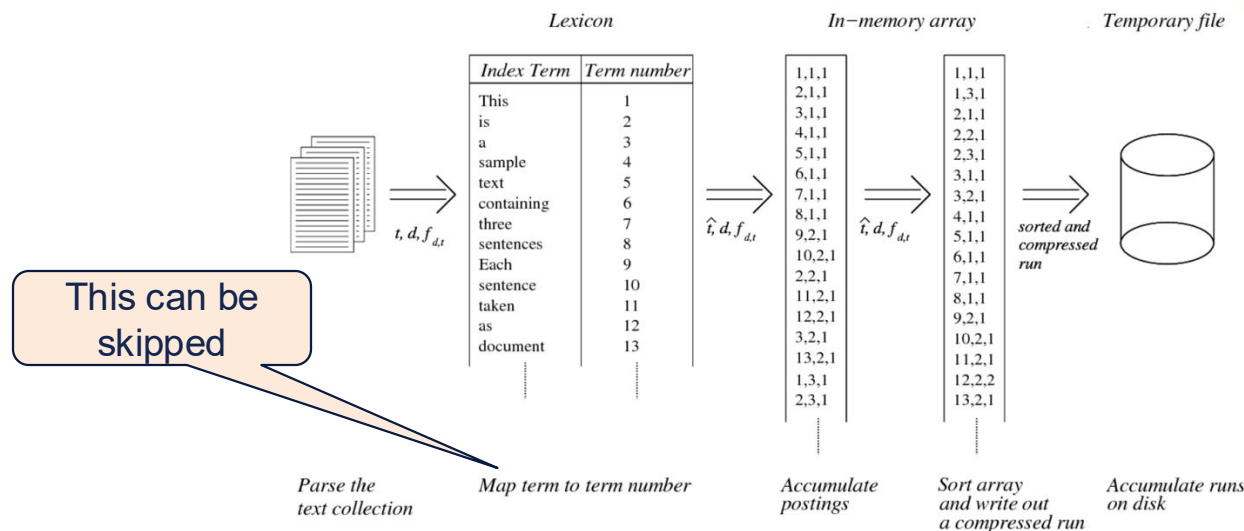
Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Sort-based index construction

- ❖ As we build the index, we parse docs one at a time
 - The final postings for any term are incomplete until the end
- ❖ At 12 bytes per non-positional postings entry (termID, docID, freq), demands a lot of space for large collections
 - $T = 100,000,000$ in the case of RCV1 (1.2GB)
 - So ... we can do this in memory, but typical collections are much larger.
- ❖ How can we construct an index for very large collections?
 - Taking into account hardware constraints . . .
 - Memory, disk, speed, etc.
- ❖ We need to store intermediate results on disk
 - BSBI: Blocked sort-based Indexing
 - SPIMI: Single-pass in-memory indexing

BSBI: Blocked Sort-Based Indexing

- ❖ The core idea of BSBI is to divide the massive task of index construction into smaller, manageable subtasks by utilizing a two-phase process:
 - **Blocking** - Accumulate postings for each block, sort, write to disk
 - **Merging** - Then merge the blocks into one long sorted order



BSBI - Blocking (Indexing in Memory)

- ❖ Process the document collection in sequential blocks
- ❖ Read a Block
 - The system reads a sequential chunk of documents (a "block") from the disk. The size of this block is determined by the available main memory.
- ❖ Generate $\langle \text{term}, \text{docID} \rangle$ pairs
 - For all documents in the block, the system performs the standard text processing steps (tokenization, stemming, stopping) and generates a list of $\langle \text{term}, \text{docID} \rangle$ pairs.
- ❖ Sort the pairs
 - These pairs are sorted, typically alphabetically by term, and secondarily by docID.
- ❖ Write to Disk (Partial Index)
 - The sorted list is written to disk as a partial (or intermediate) index file. This file contains the complete postings for only a subset of the documents, but it is organized optimally for the next phase.

BSBI - Merging to Final Index

- ❖ The second phase combines all the sorted partial index files into a single, final Inverted Index.
- ❖ K-way Merge
 - The system performs a multi-way merge (or K-way merge) operation, where K is the number of partial index files.
- ❖ Sequential Read
 - The merge algorithm reads one item (the next $\langle \text{term}, \text{docID} \rangle$ pair) sequentially from the beginning of each of the K partial index files.
- ❖ Combine Postings
 - Because the files are all sorted by term, the algorithm can easily combine all postings for the same term and write them out to the final Inverted Index file on the disk. For example, all "computer" postings from file 1, file 2, ..., file K are grouped together.

BSBI - Advantages

❖ Scalability

- BSBI effectively handles collections of any size, limited only by available disk space, making it a highly scalable solution for very large corpora.

❖ Memory Efficiency

- It bypasses the limitation of main memory by only requiring enough RAM to store and sort one partial block at a time, or to manage the buffers for the multi-way merge.

❖ Sequential I/O

- The sorting and merging process relies heavily on sequential disk reads and writes, which are much faster than random disk access.

❖ BSBI is a foundational technique in IR

- Replaced by Sort-Based Indexing (SPI) methods in **distributed environments** (like MapReduce), which manage the sorting and merging of massive datasets more efficiently.

SPIMI: Single-Pass In-Memory Indexing

- ❖ The core distinction is that SPIMI **avoids the explicit sorting step** required by BSBI, significantly streamlining the process.
 - SPIMI operates in a single, sequential pass over the document collection
- ❖ **1. Sequential Processing and Block Generation**
 - Read a Block
 - The system reads a sequential block of documents from the corpus, similar to BSBI.
 - In-Memory Index
 - Instead of generating simple term-docID pairs, SPIMI immediately **builds a local Inverted Index structure in memory for the documents in that block**. This index consists of a local dictionary and its associated posting lists.
 - Check Memory
 - The process continues until the memory allocated for the current block's index is full.

SPIMI: Single-Pass In-Memory Indexing

❖ 2. Output and Persistence

– Freeze

- When the memory is full, the system "freezes" the local in-memory index.

– Write to Disk

- The complete local index is written to disk. The key is that because the index was built in-memory, the postings for each term are already sorted by Document ID within that block.

– Unique Naming

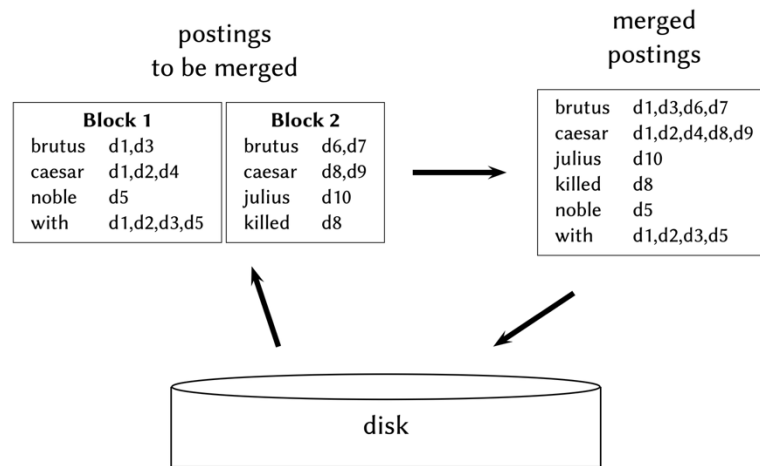
- Each written block is given a unique name (e.g., Block 1, Block 2, ...) to prevent collisions between terms.

❖ 3. Final Merging

- After all documents are processed, the system has several unique, sorted index files on disk. The final index is created by performing a simple multi-way merge of these blocks.
- The merge operation is straightforward because the postings within each block are already sorted by DocID.

How to merge the sorted runs?

- ❖ Can do binary merges
 - For 10 blocks, merge tree will have 4 layers ($\log_2 10$)
- ❖ During each layer, read into memory runs in blocks of 10M, merge, write back



- ❖ It is more efficient to do a n-way merge, where you are reading from all blocks simultaneously
 - Reading chunks of each block into memory and then write out an output chunk

SPIMI - Advantages Over BSBI

❖ No Global Sorting

- The major gain is that SPIMI avoids the costly global sort of millions of $\langle \text{term}, \text{docID} \rangle$ pairs required by BSBI. The sorting is implicitly handled at the local, in-memory block level.

❖ Memory Efficiency

- It's designed to be memory-efficient by ensuring that only one block's index is ever resident in RAM at a time.

❖ Streamlined Index Construction

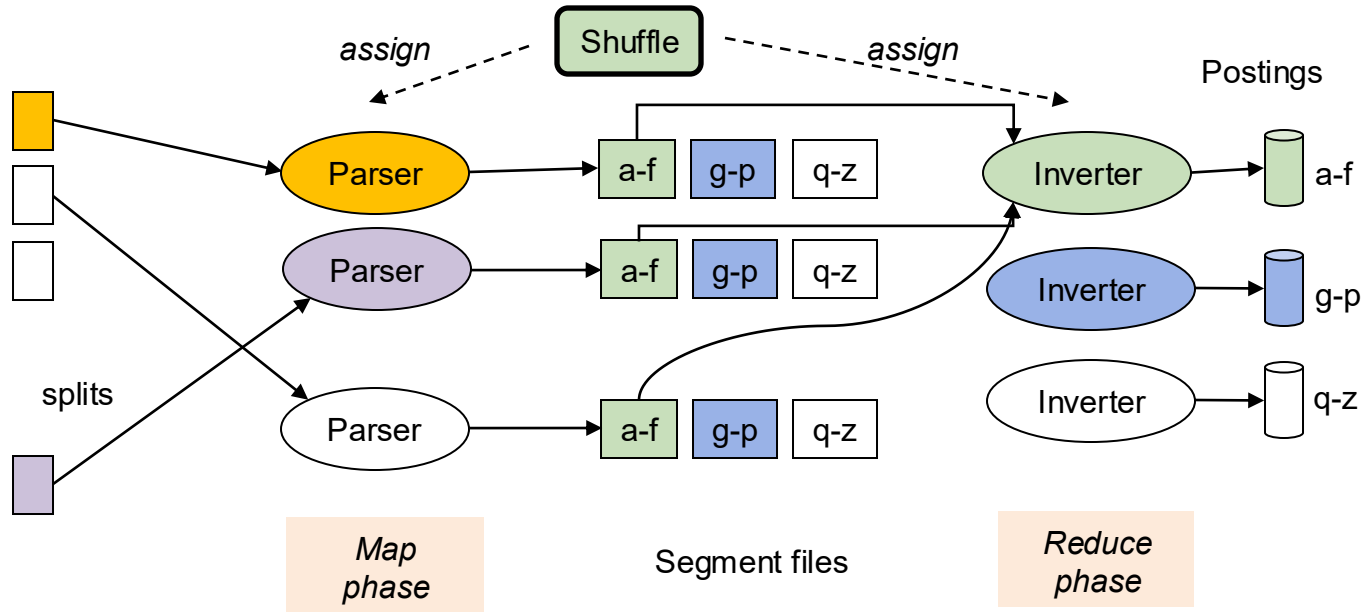
- It's a faster, single-pass approach to generating the initial partial indices.

❖ SPIMI is highly effective when the vocabulary is large, but the available memory is sufficient to hold the index created from a few thousand documents (a block).

Distributed indexing

- ❖ Distributed processing is driven by the need to index and analyze huge amounts of data
 - For large-scale indexing (e.g. web-scale)
- ❖ Quite common in some NoSQL databases
 - E.g., MongoDB, Cassandra
- ❖ Large numbers of inexpensive servers used rather than larger, more expensive machines
 - How do we exploit such a pool of machines?

Data flow



MapReduce

❖ MapReduce is a distributed programming tool designed for indexing and analysis tasks:

- Map (parallel document processing)

- Count the terms in each document (tokenize, stem, ...)
- Produces (document, count) pairs

- Shuffle

- Send all pairs for the same term to the same reducer

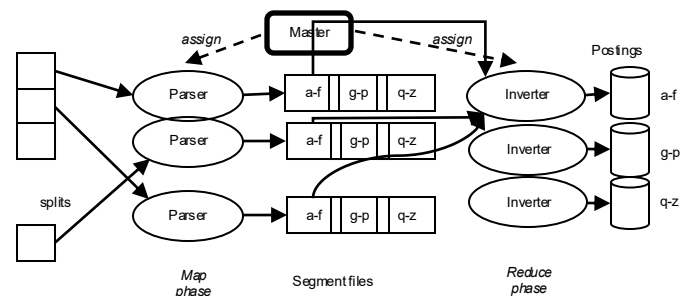
- Reduce (parallel term processing)

- Build the postings file

❖ Fault-tolerant:

- If a machine fails to complete, that task is restarted

❖ The Mapper or Reducer is called multiple times on the same input, the output will always be the same



MapReduce

- ❖ Index construction was just one phase.
- ❖ Another phase: transforming a term-partitioned index into a document-partitioned index.
 - Term-partitioned: one machine handles a subrange of terms
 - Document-partitioned: one machine handles a subrange of documents
- ❖ Most search engines use a document-partitioned index ... better load balancing, etc.

Schema for index construction in MapReduce

❖ Schema of map and reduce functions

- map: input $\rightarrow$ list(k, v)
- reduce: (k,list(v)) $\rightarrow$ output

❖ map:

- collection $\rightarrow$ list(termID, docID)

d1 : Caesar came, Caesar conquered.

d2 : Caesar died.

$\rightarrow$

< Caesar,d1>, <came,d1>, < Caesar,d1>, <conquered,d1>, < Caesar,d2>,
<died,d2>

❖ reduce:

- (<termID1, list(docID)>, <termID2, list(docID)>, ...) $\rightarrow$ (postings list1, postings list2, ...)

(<Caesar, (d1,d2,d1)>, <died, (d2)>, <came, (d1)>, <conquered, (d1)>)

$\rightarrow$

(<Caesar, (d1:2,d2:1)>, <died, (d2:1)>, <came, (d1:1)>,
<conquered, (d1:1)>)

MapReduce includes a shuffle/combining stage between map and reduce

Dynamic indexing

- ❖ Up to now, we have assumed that collections are static
- ❖ They rarely are:
 - Documents come in over time and need to be inserted
 - Documents are deleted and modified
- ❖ This means that the dictionary and postings lists have to be modified:
 - Postings updates for terms already in the dictionary
 - New terms added to the dictionary

Simplest approach

- ❖ Maintain “big” main index
- ❖ New docs go into “small” auxiliary index
- ❖ Search across both, merge results
- ❖ Deletions
 - Invalidation bit-vector for deleted docs
 - Filter docs output on a search result by this invalidation bit-vector
- ❖ Periodically, re-index into one main index

Issues with main and auxiliary indexes

- ❖ The problem of frequent merges
 - We touch stuff a lot
 - Poor performance during the merge
- ❖ Actually:
 - Merging the auxiliary index into the main index is efficient if we keep a separate file for each postings list.
 - Merge is the same as a simple append.
 - But then we would need a lot of files – inefficient for O/S.
- ❖ In reality:
 - Use a scheme somewhere in between (e.g., split very large postings lists, collect postings lists of length 1 in one file, etc.)

Logarithmic merge

- ❖ Maintain a series of indexes, each twice as large as the previous one
- ❖ Keep smallest (Z_0) in memory
- ❖ Larger ones ($I_0, I_1, \dots$) on disk
- ❖ If Z_0 gets too big ($> n$), write to disk as I_0
- ❖ or merge with I_0 (if I_0 already exists) to form Z_1
- ❖ Either write merged Z_1 to disk as I_1 (if no I_1)
- ❖ Or merge with I_1 to form Z_2
- ❖ ...

Logarithmic merge

- ❖ Main and Auxiliary index
 - Index construction time is $O(T^2)$ as each posting is touched in each merge
- ❖ Logarithmic merge
 - Each posting is merged $O(\log T)$ times, so complexity is $O(T \log T)$
- ❖ So logarithmic merge is much more efficient for index construction
 - But query processing now requires the merging of $O(\log T)$ indexes
 - Whereas it is $O(1)$ if you just have a main and auxiliary index

Summary

- ❖ Index construction
- ❖ Data structures
- ❖ Indexing strategies
- ❖ Distributed indexing

